2024 IEEE Intelligent Vehicles Symposium (IV)
June 2-5, 2024. Jeju Island, Korea

A Safe Reinforcement Learning driven Weights-varying Model Predictive Control for Autonomous Vehicle Motion Control

Baha Zarrouki[1,2], Marios Spanakakis[2] and Johannes Betz[2]

2024 IEEE Intelligent Vehicle Symposium (IV) | 979-8-3503-4881-1/24/$31.00 ©2024 IEEE | DOI: 10.1109/IV55156.2024.10588747

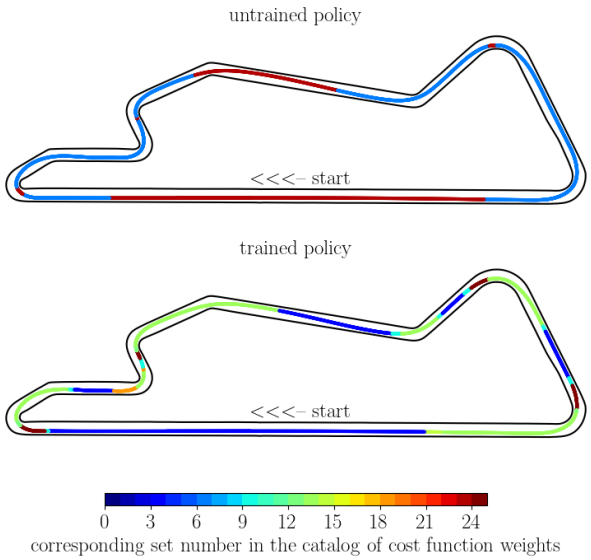
***Abstract*— Determining the optimal cost function parameters of Model Predictive Control (MPC) to optimize multiple control objectives is a challenging and time-consuming task. Multi-objective Bayesian Optimization (BO) techniques solve this problem by determining a Pareto optimal parameter set for an MPC with static weights. However, a single parameter set may not deliver the most optimal closed-loop control performance when the context of the MPC operating conditions changes during its operation, urging the need to adapt the cost function weights at runtime. Deep Reinforcement Learning (RL) algorithms can automatically learn context-dependent optimal parameter sets and dynamically adapt for a Weights-varying MPC (WMPC). However, learning cost function weights from scratch in a continuous action space may lead to unsafe operating states. To solve this, we propose a novel approach limiting the RL action space within a safe learning space that we represent by a catalog of pre-optimized feasible BO Pareto-optimal weight sets. We conceive an RL agent not to learn in a continuous space but to select the most optimal discrete actions, each corresponding to a single set of Pareto optimal weights, by proactively anticipating upcoming control tasks in a context-dependent manner. This approach introduces a two-step optimization: (1) safety-critical with BO and (2) performance-driven with RL. Hence, even an untrained RL agent guarantees a safe and optimal performance. Simulation results demonstrate that an untrained RL-WMPC shows Pareto-optimal closed-loop behavior and training the RL-WMPC helps exhibit a performance beyond the Pareto-front. The code used in this research is publicly accessible as open-source software:** `https://github.com/bzarr/TUM-CONTROL`

I. Introduction

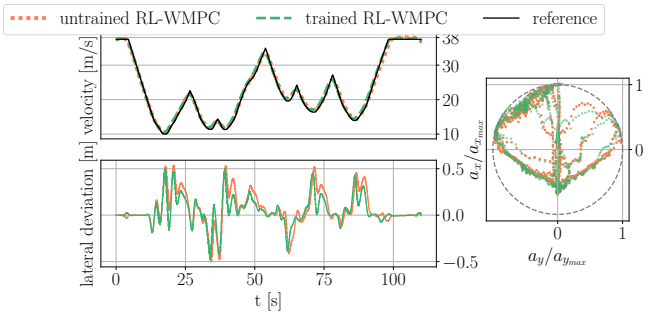
The performance of an MPC heavily relies on the formulation of its cost function, especially the appropriate selection of cost function weights, a pivotal aspect of MPC design. This process poses three key challenges: it demands significant manual effort and domain expertise, making it time-consuming and susceptible to suboptimality; the cost function profoundly influences operational safety, as unsuitable weight sets render the control problem infeasible; and a given weight distribution is valid only for a specific operating point, with changes in operating conditions potentially causing substantial performance degradation. This safety concern is particularly relevant for systems like autonomous vehicles, operating across diverse conditions.

[1] Chair of Automotive Technology, [2] Professorship of Autonomous Vehicle Systems, TUM School of Engineering and Design, Technical University of Munich; Munich Institute of Robotics and Machine Intelligence (MIRMI), {baha.zarrouki, m.spanakakis, johannes.betz}@tum.de

This work is a result of the joint research project ATLAS-L4, supported by the German Federal Ministry for Economic Affairs and Climate Action.



(a) The RL agent dynamically adjusts MPC cost function weights online to align with an optimal raceline trajectory on the Monteblanco racetrack. The RL agent selects a set (represented by a certain discrete color) from a pre-computed catalog of Pareto optimal weights for each switching time. The sets (0-10) are optimized for straight segments, (15-25) for curves, and (11-14) strike a tradeoff between the two.



(b) Velocity-, lateral deviation and the gg-diagram plots.

Fig. 1: Safe RL driven Weights-varying MPC: comparison between an untrained- and a trained policy. Even the untrained RL policy shows good closed-loop behavior, attributed to the global safety inherent in the pre-computed weights catalog obtained through Multi-objective Bayesian Optimization (MOBO).

Approaches in the literature address these issues through controller design automation. In [1], multi-objective Genetic Algorithms (GA) are used to determine MPC weights, en-

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 01,2026 at 15:46:12 UTC from IEEE Xplore. Restrictions apply.

hancing closed-loop performance. Additionally, [2] presents a GA approach that handles constraints. Furthermore, collaborative Particle Swarm Optimization (PSO) is employed in several instances to navigate the search space and identify tuning parameters that enhance trajectory tracking performance, particularly in the context of MPCs controlling quadrotors [3]. Another class of algorithms employed for automatic MPC tuning is Bayesian Optimization (BO). [4] and [5] propose performance-driven MPC tuning strategies using BO, while [6], [7], and [8] utilize constrained BO in their respective works. Additionally, [9] uses a multi-objective BO approach to mitigate plant-model mismatch. Moreover, Reinforcement Learning (RL) proves valuable in learning controller design. In [10], an approach is presented to automatically tune Stochastic MPC meta-parameters online using Deep RL, enhancing uncertainty handling, constraints robustification, feasibility, and closed-loop performance. [11] employs RL algorithms to determine the prediction horizon of MPC, reducing computational complexity, while [12] focuses on learning other MPC meta-parameters. Additionally, [13] introduces RL for the automatic tuning and determination of a static set of weights for an MPC controlling a quadrotor. Meanwhile, [14][15] present a Weights-varying Nonlinear MPC driven by Deep RL, enabling the automatic learning and online adaptation of cost function weighting matrices to optimize multi-control objectives. However, the challenge arises when learning cost function weights in a continuous action space, as safety is not inherently guaranteed. Our approach tackles this concern by constraining the action space to discrete options, allowing the agent to select sets from a catalog of pre-optimized feasible weights determined by Bayesian Optimization (BO). In [16], safety considerations are addressed when integrating Reinforcement Learning (RL) with Model Predictive Control (MPC) within a resulting Bayesian-MPC framework. In [17][18], the emphasis is on optimizing control law parameters with a safety guarantee. This approach models the performance measure as a Gaussian process and explores new controller parameters, specifically those with a high probability of performance above a safe threshold. However, these works do not specifically tackle finding optimal cost function weights.

In summary, our work contributes in three key ways:

- We conceive a Weights-varying MPC (WMPC) driven by a Safe RL agent. The RL agent uses a look-ahead design to adjust MPC cost function weights autonomously under varying operating conditions, selecting the most suitable set from a Pareto front optimized through Multiobjective Bayesian Optimization (MOBO).
- We demonstrate the adaptability of our approach in dynamically adjusting the weights of a Nonlinear MPC for controlling a full-scale autonomous vehicle to follow an optimal raceline in simulation. Additionally, we provide experimental safety evidence, showcasing Pareto optimality even with an untrained agent.
- We perform a context-dependent analysis of the RL agent's decision-making process and assess our approach's generalization and robustness capabilities through testing in unseen environments. We also explore the impact of continuous learning in an inexperienced environment.

II. Weights-varying MPC

Weights-varying MPC (WMPC) [14] is an MPC that adapts its cost function weights during online operation. Different weights optimized for several operation conditions enhance the overall closed-loop performance and show advantages compared to an MPC operating with a static cost function weights set. A Weights scheduler module provides the MPC with a new set of weights computed by a weights optimization policy. The Weights scheduler can be designed to update the MPC event-based or following a schedule.

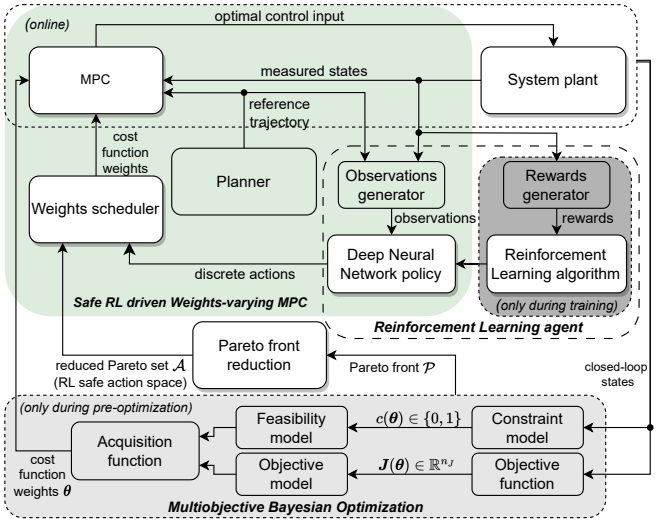


Fig. 2: Architecture of the Safe RL driven Weights-varying MPC

In this work, we advocate a Deep Neural Network (DNN) policy to determine optimal cost function weights following a fixed schedule based on observations of relevant closed-loop states and future MPC reference trajectories. We conceive the DNN policy to choose every switching time T_{sw} a cost function weights set from a catalog of pre-optimized safe weights. This catalog represents a Pareto set $\mathcal{A}$ that we determine with an offline Multi-Objective Bayesian Optimization (Sec.IV). The set $\mathcal{A}$ contains cost function weights locally optimized for single operating conditions. Yet, each set is valid for a global safe operation, i.e., for an MPC with static weights.

During training, a DNN policy offline is learned with a Deep Reinforcement Learning (RL) algorithm (Sec.V). Each RL action is safe as the agent does not learn cost function weights, but automatically learns which optimal cost function weights from the catalog $\mathcal{A}$ are optimal depending on the context and when to switch them. Thus, we conceive a Safe Deep RL driven WMPC. Figure 2 denotes its architecture. Note that safe in this context refers to safe learning and not safe MPC or safe control.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 01,2026 at 15:46:12 UTC from IEEE Xplore. Restrictions apply.

III. Static Nonlinear MPC

We consider the following static Nonlinear MPC (NMPC) where, $\boldsymbol{x} \in \mathbb{R}^{n_x}$ denotes the state vector, $\boldsymbol{u} \in \mathbb{R}^{n_u}$ the control vector, t the discrete time, T_p the prediction horizon, f the system dynamics, $\boldsymbol{h}$ and $\boldsymbol{h}^e$ the path and terminal inequality constraints on states and control inputs, which can be linear or nonlinear, and $\boldsymbol{x}_0$ the initial state.

Problem 1 Nominal NMPC

$$\begin{aligned} \min_{\boldsymbol{x}(.),\boldsymbol{u}(.)} \quad & \int_{\tau=0}^{T_p} l(\boldsymbol{x}(\tau),\boldsymbol{u}(\tau))d\tau \\ & + m(\boldsymbol{x}(T_p)) \\ \text{subject to} \quad & \boldsymbol{x}(0) = \boldsymbol{x}_0, \\ & \dot{\boldsymbol{x}}(t) = f(\boldsymbol{x}(t),\boldsymbol{u}(t)), \quad t \in [0,T_p), \\ & \boldsymbol{h}(\boldsymbol{x}(t),\boldsymbol{u}(t)) \leq \bar{\boldsymbol{h}}, \quad t \in [0,T_p), \\ & \boldsymbol{h}^{\mathrm{e}}(\boldsymbol{x}(T_p)) \leq \bar{\boldsymbol{h}}^{\mathrm{e}}, \end{aligned} \tag{1}$$

For the cost function, we employ a nonlinear least squares approach for the stage cost ($l : \mathbb{R}^{n_{\mathrm{x}}} \times \mathbb{R}^{n_{\mathrm{u}}} \to \mathbb{R}$) and terminal cost ($m : \mathbb{R}^{n_{\mathrm{x}}} \to \mathbb{R}$):

$$\begin{aligned} l(\boldsymbol{x},\boldsymbol{u}) =& \frac{1}{2}\|\boldsymbol{y}(\boldsymbol{x},\boldsymbol{u}) - \boldsymbol{y}_{\mathrm{ref}}\|^2_W \\ m(\boldsymbol{x}) =& \frac{1}{2}\|\boldsymbol{y}^e(\boldsymbol{x}) - \boldsymbol{y}^e_{\mathrm{ref}}\|^2_{W^e} \end{aligned} \tag{2}$$

Here, $W = \mathrm{diag}(Q,R)$ and $W_e = Q_e$ represent weighting matrices for the stage and terminal costs, respectively, where Q and Q_e are matrices used for weighting states and R for inputs. In this work, we assume $Q = Q_e$.

We consider the combined longitudinal and lateral motion control of a full-scale autonomous passenger vehicle. For the full MPC problem formulation, we refer to [19] [20]. The cost terms are defined as follows:

$$\begin{aligned} \boldsymbol{y}(\boldsymbol{x},\boldsymbol{u}) &= [x_{\mathrm{pos}}, y_{\mathrm{pos}}, \psi, v_{\mathrm{lon}}, j, \omega_f] \\ \boldsymbol{y}_{\mathrm{ref}} &= [x_{\mathrm{pos,ref}}, y_{\mathrm{pos,ref}}, \psi_{\mathrm{ref}}, v_{\mathrm{ref}}, 0, 0] \\ \boldsymbol{y}^e_{\mathrm{ref}} &= [x^e_{\mathrm{pos,ref}}, y^e_{\mathrm{pos,ref}}, \psi^e_{\mathrm{ref}}, v^e_{\mathrm{ref}}] \end{aligned} \tag{3}$$

And the weighting matrices:

$$\begin{aligned} Q &= \mathrm{diag}(q_{x,y}, q_{x,y}, q_\psi, q_v) \\ R &= \mathrm{diag}(r_j, r_\omega) \end{aligned} \tag{4}$$

Here, x_{pos} and y_{pos} represent the x- and y-coordinates of the ego vehicle, ψ the yaw angle, v_{lon} the longitudinal velocity, j the longitudinal jerk, and ω_f the steering rate at the front wheel. Moreover, we employ two slack variables, $L1$ for a linear- and $L2$ for a quadratic constraint violation penalization term, relaxing the constraints and helping the solver find a solution.

This work aims to automatically learn and adapt the weighting matrices Q and R, and the slack terms $L1$ and $L2$.

IV. Multi-objective Bayesian Optimization: Determining Pareto Optimal Weight Sets

We require a safe action space to allow for Safe RL as described above. This is obtained by identifying optimal, feasible weight sets within the defined parameter space. The tuning task is formulated as a multiobjective optimization problem and solved using Multiobjective Bayesian Optimization (MOBO) to obtain the set of Pareto optimal weight sets.

A. Formulating the optimization problem

For the given control design task, the decision variables are the cost function weights and soft constraint penalty terms:

$$\boldsymbol{\theta} = [q_{x,y},\ q_\psi,\ q_v,\ r_j,\ r_\omega,\ L_1,\ L_2] \tag{5}$$

In the optimization process, the decision variables are fine-tuned to minimize two distinct performance metrics: the vehicle's lateral deviation from the reference trajectory and the deviation from the reference velocity. Specifically, the lateral deviation metric focuses on the maximum observed deviation throughout the entire trajectory:

$$J_0(\boldsymbol{\theta}) = \max\left(e_{\mathrm{lat,0}}, e_{\mathrm{lat,1}}, \ldots, e_{\mathrm{lat,N}}\right) \tag{6}$$

Here, $e_{\mathrm{lat,k}}$ denotes the lateral deviation at step k. The velocity deviation metric is defined as:

$$J_1(\boldsymbol{\theta}) = \sqrt{\frac{1}{N}\sum_{k=0}^{N}\left(v_k(\boldsymbol{\theta}) - v_{\mathrm{ref,k}}\right)^2}. \tag{7}$$

This metric is formulated as a root-mean-square (RMS) error. To ensure solution feasibility, i.e., the compliance to physical constraints and the absence of intolerable behavior, each solution must satisfy two global constraint conditions:

1) A feasible parameter set must not violate the MPC's constraints, e.g., maximum allowed combined lateral and longitudinal accelerations.
2) The lateral deviation must not exceed a defined maximum value $e_{\mathrm{lat,max}}$.

Consequently, we formulate the optimization problem as:

Problem 2 Multiobjective Bayesian Optimization

$$\begin{aligned} \min_{\boldsymbol{\theta}\in\mathbb{R}^{n_\theta}} \quad & \boldsymbol{J}(\boldsymbol{\theta}) = [J_1(\boldsymbol{\theta}), \ldots, J_{n_J}(\boldsymbol{\theta})] \\ \text{subject to} \quad & \boldsymbol{\theta}_{lb} \leq \boldsymbol{\theta} \leq \boldsymbol{\theta}_{ub} \\ & \boldsymbol{g}(\boldsymbol{\theta}) \leq \bar{\boldsymbol{g}} \end{aligned} \tag{8}$$

Where $\boldsymbol{J}$ denotes the vector containing the optimization objectives and $\boldsymbol{g}$ the optimization constraint functions. $\boldsymbol{\theta}_{lb}$ and $\boldsymbol{\theta}_{ub}$ denote the individual boundaries for the decision variables.

B. Surrogate models

The surrogate models approximate the real, unknown objective function and serve to select the most promising point to sample next, i.e., a parameter set that is likely to be beneficial for finding global optima. The surrogate model is constructed from a set of three Gaussian Processes (GPs) for the given system. Two of the GPs model the expected outcome for each optimization objective. The third GP is a binary classifier that models the probability of feasibility for each point in the decision space. This feasibility model utilizes the information gained from system failures and crashes due to unfavorable weight sets. Each model uses a

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 01,2026 at 15:46:12 UTC from IEEE Xplore. Restrictions apply.

constant mean function and the Radial Basis Function (RBF) kernel. The third GP model transforms the classification task into a regression task by leveraging a modified Dirichlet likelihood as introduced in [21]. Given a configuration θ, the model outputs a probability distribution describing the expected solution feasibility.

C. Acquisition function

The acquisition function encapsulates the strategy for selecting promising sample points, which is decisive for the sample efficiency of BO. Various acquisition functions have been proposed for multiobjective optimization, of which the most established is the Expected Hypervolume Improvement (EHVI) [22]. This function bases the search strategy on the volume in objective space enclosed by the Pareto front and a reference point R. Essentially, the EHVI acquisition function serves as a decision-making criterion, favoring candidate solutions that offer the most significant expected improvement to the multidimensional objective space, thereby steering the optimization process towards regions with high potential for Pareto front expansion. However, the EHVI acquisition function itself does not provide means to include the feasibility estimate of the third surrogate model in the decision process. To take solution feasibility into account, the feasibility estimate obtained from the feasibility model is used as a weight for the acquisition function.
The acquisition value $\alpha(\vec{\theta})$ for any given $\vec{\theta}$, i.e., for any parameter space possible configuration, as well as the feasibility weight $\alpha_{\text{feas}}(\vec{\theta})$ are defined as:

$$\begin{aligned} \alpha(\vec{\theta}) &= \alpha_{\text{EHVI}}(\vec{\theta}) \ \cdot \ \alpha_{\text{feas}}(\vec{\theta}) \\ \alpha_{\text{feas}}(\vec{\theta}) &= \min\left(\hat{\mu}_{\text{feas}}\left(\vec{\theta}\right)^{k} \ + \ \epsilon \cdot \hat{\sigma}_{\text{feas}}(\vec{\theta}),\ 1\right) \end{aligned} \tag{9}$$

The latter is calculated based on $\hat{\mu}_{\text{feas}}(\vec{\theta})$ and $\hat{\sigma}_{\text{feas}}(\vec{\theta})$, which denote the mean and standard deviation of the feasibility estimate. The parameter k provides a mean to control the degree of trust in the probability estimate interpretation. For $k > 1$, estimated probability values below 1 are skewed towards 0, encouraging sampling for highly likely feasible configurations. Conversely, for $k < 1$, smaller feasibility estimates are biased towards 1, increasing the likelihood of testing such configurations. Scaling $\hat{\sigma}_{\text{feas}}(\vec{\theta})$ by ϵ offers control over the exploration degree and encourages sampling in regions lacking reliable feasibility estimates. During all experiments, the feasibility weight parameters have been set to $k = 1$ and $\epsilon = 0.8$. It is important to note that the feasibility weight is capped at 1. While the weighting can reduce the likelihood of sampling unfeasible regions, it does not have an additional influence on the selection process in regions expected to be feasible.

D. Training track segmentation

To specialize parameterizations for distinct track features, our approach involves segmenting training tracks based on curvature profiles. MOBO evaluates control performance separately for each segment group, allowing the distinction between parameter sets excelling in curved sections and those performing well on straight track segments. This results in two parallel optimization processes seeking the best performance on their respective trajectory types. Note that a specialized parameterization is considered only if it respects the global feasibility condition.

E. Optimization algorithm

The optimization framework concurrently optimizes for two scenarios using a parallel approach. Each segment group's parameterization is independently tested, producing two distinct datasets for performance on straight and curved sections. In each iteration, a sample point is selected alternately from one of the datasets. The surrogate models are fitted to the corresponding dataset, considering joint constraint fulfillment. The strategy in Algorithm 1 enhances efficiency by sharing information between the two optimization processes, ensuring that a parameter set inducing unfeasible behavior in one scenario is deemed non-viable, regardless of performance in the other. This parallel search generates two closely connected Pareto fronts, facilitating the sharing of entries.

Algorithm 1 Parallel constrained Bayesian optimization for multiple evaluation environments

Output: Set of non-dominated solutions $\mathcal{P}$
1: Obtain initial data set $\mathcal{D} \leftarrow \{\vec{\theta}_i,\ \vec{J}(\vec{\theta}_i),\ c(\vec{\theta}_i)\}$ for $i = 0, \ldots, n_{\text{init}}$
2: **for** $k \in (1, \ldots, N)$ **do**
3: Fit objective model to $\vec{J}(\vec{\theta}_i)$
4: Fit constraint model to $c(\vec{\theta}_i)$
5: Obtain next sample point $\hat{\vec{\theta}}_k \leftarrow \operatorname{argmax} \alpha(\vec{\theta})$
6: Evaluate objective function $\vec{f}(\hat{\vec{\theta}}_k)$
7: Augment data set $\mathcal{D}_k \leftarrow \{\hat{\vec{\theta}}_k, \vec{J}(\vec{\theta}_k),\ c(\vec{\theta}_k)\}$
8: **end for**

F. Pareto front reduction

The optimization process yields dense Pareto fronts. Solution clusters may emerge from convergence, impacting RL by favoring certain behaviors. To address this, reducing the number of actions is beneficial, preventing biases while maintaining strategy diversity. Balancing action pruning is crucial to avoid compromising the exploration of available parameterizations. We employ a reduction technique based on relative distances in the objective space to condense the action space while maintaining diversity among behaviors (Algorithm 2). We define the reduced Pareto front $\mathcal{A}$. First, the minima along each dimension, representing the best global performance for each objective, are added to the set $\mathcal{A}$. Then, the remaining parameterizations are grouped using k-means clustering to extract a representative subset. This iterative process develops clusters with similar characteristics, and the center of each cluster, representing a behavior, is added to $\mathcal{A}$.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 01,2026 at 15:46:12 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2 Pareto front reduction with k-means clustering

Input: Dense n_J dimensional Pareto front composed of $n_{\mathcal{P}}$ points: $\mathcal{P} = \{p_0, \ldots, p_{n_{\mathcal{P}-1}}\}$ with $p_i \in \mathbb{R}^{n_{\boldsymbol{\theta}}}\ \forall i \in [0, \ldots, n_{\mathcal{P}-1}]$

Output: Reduced set of $n_{\mathcal{A}}$ equally-distributed Pareto points $\mathcal{A} = \{\boldsymbol{\theta}^{(0)}, \ldots, \boldsymbol{\theta}^{(n_{\mathcal{A}}-1)}\}$ with $\boldsymbol{\theta}^{(i)} \in \mathbb{R}^{n_{\boldsymbol{\theta}}}\ \forall i \in [0, \ldots, n_{\mathcal{A}} - 1)]$

Min-max normalize $\mathcal{P}$
$\mathcal{A} \leftarrow \emptyset$
for $i \in \{1, ..., n_J\}$ **do**
    $p_{\min,i} \leftarrow \arg\min_{p \in \mathcal{P}}\{p[i]\}$
    $\mathcal{A} \leftarrow \mathcal{A} \cup \{p_{\min,i}\}$
end for
$\{C_1, C_2, \ldots, C_{n_{\mathcal{A}}-n_{\boldsymbol{\theta}}}\} \leftarrow$ k-Means$(\mathcal{P}, n_{\mathcal{A}} - n_J)$
for $i \in \{1, ..., n_{\mathcal{A}} - n_J\}$ **do**
    $\mathcal{A} \leftarrow \mathcal{A} \cup \{C_i\}$
end for

V. Deep Reinforcement Learning: Safe Learning Weights-varying MPC

In this section, we introduce our Weights-varying MPC (WMPC), designed to dynamically adjust cost function weights for optimal closed-loop performance in response to changing environments and driving tasks. Utilizing a Deep Neural Network (DNN) policy trained through state-of-the-art Deep RL (DRL) algorithms, we illustrate the architecture of our DRL-driven WMPC in Fig.2. The DNN policy computes new actions (Sec.V-A), and the parameter scheduler adjusts WMPC parameters at predefined switching time instances $T_{sw} = n \cdot T_{\text{s,sim}}, n \in \mathbb{N}$, where $T_{\text{s,sim}}$ is the environment discretization time. New actions are computed via a feed-forward step through the DNN using observation vector stacks generated from current measured states and the future reference trajectory (Sec.V-C).

During the training/learning phase, the Proximal Policy Optimization (PPO) method [23] updates the DNN policy based on rewards collected (Sec.V-B) after a specified number of steps in the environment n_{steps}, i.e., new updated WMPC parameters steps. Following training, the rewards generator and the RL algorithm are no longer necessary, and the architecture remains unchanged for deployment. Figure 3 outlines the architecture of the DNN policy tailored for this problem.

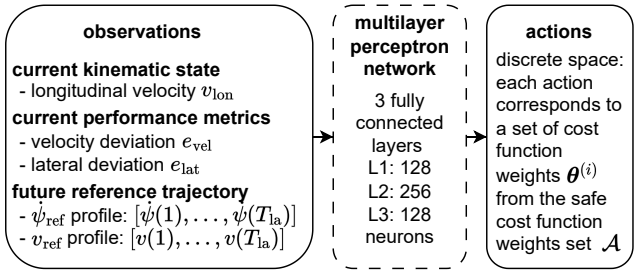


Fig. 3: Architecture of the Deep Neural Network driven WMPC

A. Defining the action space

We design a discrete action space. Each action a corresponds to one set of cost function weights $\boldsymbol{\theta}^{(i)}$ from the pre-optimized Pareto-optimal set $\mathcal{A}$, where $i \in [0, \ldots, n_{\mathcal{A}}]$. Each action is an integer $a \in \{0, \ldots, n_{\mathcal{A}}-1\}$, where $n_{\mathcal{A}} \in \mathbb{N}$ represents the number of possibilities to choose an action from, which corresponds to the number of Pareto points available in the safe space $\mathcal{A}$.

B. Designing the reward function

The reward function defines the desired behavior exhibited by the agent. In this context, our objective is to incentivize the agent to minimize lateral and velocity deviations. Works [14], [15] introduce a multi-objective Gaussian (MOG) reward function. The latter defines an attraction region and pushes the agent to converge to a certain attraction point where the optimization objectives reach their maximum value:

$$R(z_0, \ldots, z_n) = A \cdot \exp\left(-\sum_{k=0}^{n} \frac{(z_k - z_{t,k})^2}{2\sigma_k^2}\right) \quad (10)$$

Here, $[z_0, \ldots, z_n]$ denote the set of optimization objectives, $[z_{t,0}, \ldots, z_{t,n}]$ the respective target values, $[\sigma_0, \ldots, \sigma_n]$ the respective standard deviations and A the maximum achievable reward. In this work, we optimize the lateral- and velocity deviation, i.e., $z_0 = e_{\text{lat}}$ and $z_1 = e_{\text{vel}}$ with $z_{t,0} = z_{t,1} = 0$. Note that, $e_{\text{lat}} = \sqrt{\frac{1}{n}\sum_{i=1}^{n} e_{\text{lat},i}^2}$ and $e_{\text{vel}} = \sqrt{\frac{1}{n}\sum_{i=1}^{n} e_{\text{vel},i}^2}$ with $n = \frac{T_{sw}}{T_{\text{s,sim}}}$, i.e. the root mean square (RMS) of the n measured deviations between two parameter switching intervals T_{sw}. To ensure that metrics with varying orders of magnitude contribute proportionately to the overall performance evaluation, we normalize each variable to a certain normalization bounds $[a, b]$. If a metric exceeds these bounds, its value is truncated to the range [0, 1].

Thanks to the Safe RL design, there is no necessity for special conditions to penalize unintended behaviors that pose safety risks.

C. Defining the observations

Agent's observations of its environment affect its decision-making process. In our use case, we design the agent to observe the current ego velocity and to assess the performance of its last actions through lateral- and velocity deviations.

To enhance the agent's foresight capabilities, we provide it with samples from the current reference trajectory within the prediction horizon. With this design, we aim for the agent to proactively determine optimal parameters aligned with the specified objectives and according to the anticipated dynamic profiles the MPC should follow. In our use case, the agent observes future velocity and yaw rate profiles, enabling it to make informed decisions concerning future combined longitudinal and lateral behaviors. This distinction allows the agent to navigate various scenarios, such as high-speed curves and straight paths. Based on the discussions above,

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 01,2026 at 15:46:12 UTC from IEEE Xplore. Restrictions apply.

we define a general algorithm that learns and adapts the MPC cost function parameters with Safe RL (Algorithm 3).

Algorithm 3 Safe RL driven Weights-varying MPC

Set MPC parameters: T_p, T_s, T_{sw}, $L1$, $L2$, W and W_e
for $i \in \{1, ..., N\}$ **do**
  Update initial state $\boldsymbol{x}_0$ with current measurements
  Update current MPC reference
  if switching time ($i \bmod T_{\text{sw}} = 0$) **then**
    Generate observations and rewards
    Feedforward the current RL policy and get the current optimal RL action a
    Map the discrete RL action a into the corresponding weights set from the safe space $\mathcal{A}$
    Update MPC cost function weights
    if learning-step ($i \bmod n_{\text{steps}} \cdot T_{\text{sw}} = 0$) **then**
      Execute RL policy update
    end if
  end if
  Solve the MPC problem (Eq.1)
  Apply the first control input $\boldsymbol{u}_0^*$ on the real system
end for

VI. Simulation Results: Safe RL driven Weights-varying NMPC

We evaluate our concept for the combined lateral- and longitudinal motion control of a full-scaled research autonomous vehicle, a Volkswagen T7 [24]. We refer to [19] for further details about the exact problem formulation and the used models.

A. Training and simulation setup

The parameters for the MOBO framework have been set as specified in Table I. We generated 50 initial samples at random, followed by 400 iterations of BO, leading to 450 iterations in batches of 5. The acquisition function parameters have been set as specified above. We use a separate hypervolume reference point R_0 and R_1 for each segment group (Sec.IV-D).

TABLE I: Hyperparameters for the MOBO framework

Parameter	Value
N_{initial}	50
N_{BO}	400
batch size	5
k	1.0
ϵ	0.9
R_0	[0.5m, 0.75m s^{-1}]
R_1	[0.4m, 0.9m s^{-1}]

The WMPC configurations are detailed in Table III, while our NMPC implementation specifics can be found in [19]. The RL agent's training is carried out on an AMD Ryzen 7950X 5.70 GHz CPU using the Proximal Policy Optimization (PPO) algorithm implemented within the Stable Baselines3 framework [25]. Configuration details are outlined in Table II. The training process in our simulation environment spans approximately 180min on 16 parallel environments.

TABLE II: Proximal Policy Optimization hyperparameters

Parameter	Value
Learning rate [initial, final] [α_{i}, α_{f}]	[0.005, 0.0001]
Learning rate (decay) α_{decay}	0.4
Policy update every n_{steps}	512
Discount factor γ	0.8
GAE factor λ	0.98
Clipping parameter ϵ	0.2
Entropy coefficient	0.006
Minibatch size	4096
Training steps N	$1.5 \cdot 10^6$

TABLE III: Weights-varying NMPC parameters

Parameter	Value
Simulation sampling time $T_{\text{s,sim}}$	0.02s
NMPC discretization time T_s	0.08s
Prediction horizon T_p	3.04s
Weights switching time T_{sw}	1.6s
RL look-ahead horizon T_{la}	3.04s
Simulation duration per evaluation	110s

We configure the reward function (Sec.V-B) as follows: $A = 1$, the normalization bounds for the lateral- and velocity deviations are $[0, 0.4\text{m}]$ and $[0, 1\text{m s}^{-1}]$ respectively, the reward standard deviations for lateral- and velocity deviations are $\sigma_{\text{lat}} = 0.1\text{m}$ and $\sigma_{\text{vel}} = 0.5\text{m s}^{-1}$, respectively.
We train the RL agent on two real-world racetracks: Monteblanco and Modena. Reference trajectories are generated by fully exploiting the vehicle's actuation interface limits in allowed velocity and combined longitudinal and lateral accelerations (Fig.1b). Our research vehicle interface allows a maximum velocity of 37.5m s^{-1} [19]. After each training episode, the vehicle starts at a random position on one of the training racetracks. In the following sections, each evaluation consists of $110\text{s} = \frac{110}{T_{\text{s,sim}}} = 5500$ MPC steps.

B. Trajectory following performance

In Figure 1b, the closed-loop behavior of both an untrained and a trained RL agent is depicted, evaluated on the Monteblanco racetrack. Notably, both agents perform well with an advantage for the trained agent, especially w.r.t. lateral deviation. During training, the agent learns when to switch to the most optimal weights from the pre-optimized Pareto sets catalog $\mathcal{A}$.

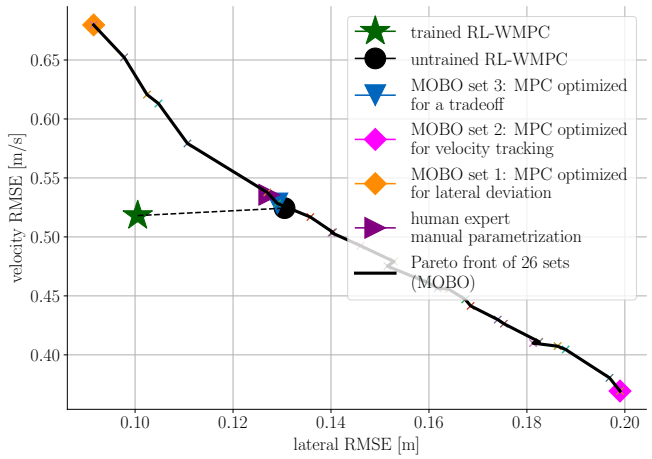
C. Analyzing Reinforcement Learning agent decisions

Figure 1a illustrates the distribution of policy decisions made by both the untrained and trained agents along the racetrack. Each discrete action corresponds to specific cost function weights in $\mathcal{A}$. The initial DNN weights' randomness influences the decisions of the untrained policy, limiting it to only a subset of the available catalog. Notably, the trained agent exhibits a successful learning trajectory. It adapts by selecting parameter sets optimized for straight segments

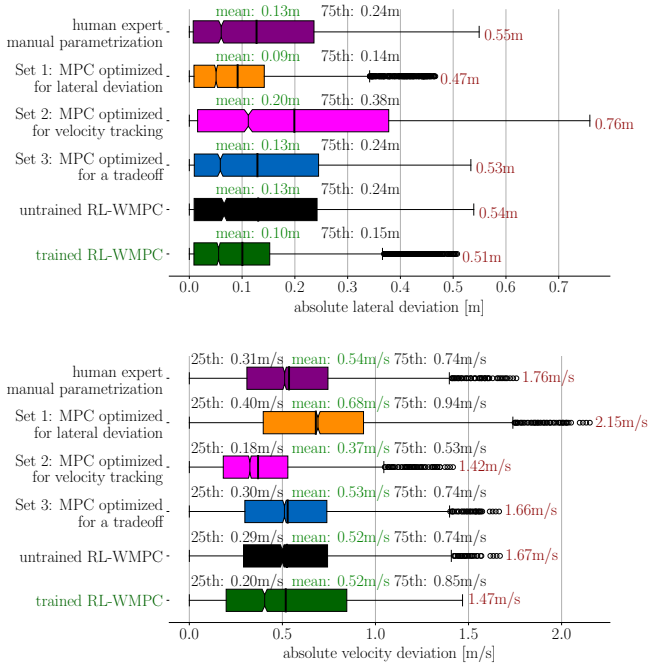
Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 01,2026 at 15:46:12 UTC from IEEE Xplore. Restrictions apply.

(numbers 0 to 10) when driving on long straights. As the track curves, the trained policy gradually transitions to sets optimized for a tradeoff (numbers 11 to 14) and then further to sets specifically tailored for curve segments (numbers 15 to 25).

D. Weights-varying MPC compared to nominal MPC



(a) Lateral- to velocity RMSE behavior: a trained RL-WMPC achieves performance beyond the Pareto-front.



(b) Box plots of the absolute lateral- and velocity deviations.

Fig. 4: Benchmark of the closed-loop performance of different settings following an optimal raceline on Monteblanco: RL driven Weights-varying MPC, nominal MPC with manual tuning and with different sets from the Pareto front.

Figure 4a showcases that dynamically varying MPC's cost function weights online by switching among different sets of the Pareto front (trained RL-WMPC) leads to a significantly improved closed-loop performance compared to relying on individual nominal MPCs parameterized with cost function sets from the Pareto front. This highlights the efficacy of our RL-WMPC approach in achieving a better tradeoff between predefined control objectives, specifically addressing lateral and velocity deviations.

The performance of the untrained RL-WMPC agent closely matches that of a nominal MPC equipped with optimal weights for a tradeoff from the Pareto front and an MPC fine-tuned by a human expert. This proves that our approach ensures a safe RL.

Figure 4b demonstrates how our trained RL-WMPC achieves a performance comparable to set 2 (notably strong in velocity tracking) w.r.t. the maximum velocity deviation. Simultaneously, it exhibits performance comparable to set 1 (particularly effective in lateral tracking) across all lateral deviation metrics.

The MPC parameterized by a human expert, the MOBO optimized for a tradeoff, and the untrained RL-WMPC all exhibit similar lateral- and velocity-tracking performances. This performance by an untrained RL-WMPC underscores the safety features inherent in the RL-WMPC approach presented in this work.

E. Generalization and robustness

To assess the generalization capabilities of our Safe RL-driven WMPC approach, we conduct evaluations on an unseen track: Las Vegas Motorspeedway (LVMS). This track was not part of the MOBO pre-optimization or RL training phases. In Figure 5, the lateral-to-velocity RMSE behavior is illustrated, similar to the representation in Figure 4a. Demonstrating consistency, the trained RL-WMPC exhibits performance that consistently surpasses the Pareto-front benchmarks. The RL-WMPC performance experiences a marginal

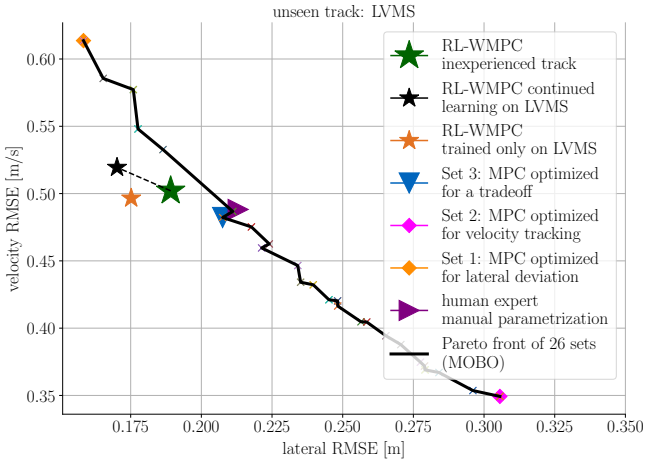


Fig. 5: Inexperienced Las Vegas Motor Speedway (LVMS): lateral- to velocity RMSE behavior.

enhancement in lateral deviation when the agent continues further learning on the new, unexplored track. However, there is a slight degradation in performance concerning velocity RMSE.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 01,2026 at 15:46:12 UTC from IEEE Xplore. Restrictions apply.

To comprehensively evaluate RL-WMPC's adaptability to an unfamiliar environment (LVMS), we benchmark its performance against an agent exclusively trained in this specific environment. Notably, the performances exhibit remarkable comparability, with a slight edge observed in favor of the latter.

VII. Conclusions and Future Work

To reduce the manual effort required to identify optimal MPC cost function weights and dynamically adapt them to changing operating conditions, we introduce a two-step automated optimization methodology. Initially, Multiobjective Bayesian Optimization (MOBO) identifies feasible Pareto optimal weight sets. Subsequently, a Look-ahead Deep Reinforcement Learning (RL) agent anticipates upcoming control tasks (reference trajectory) and learns to select the most optimal set of weights from the pre-computed Pareto front. This two-step approach results in a Safe RL driven Weights-varying MPC (WMPC).

Simulative experimental findings reveal that the trained RL-WMPC delivers closed-loop performance beyond the Pareto front, i.e., outperforms all the single sets available on the Pareto front in finding a tradeoff between the multi-optimization objectives. Simulation results show that even an untrained agent exhibits a Pareto optimal behavior, affirming the validity of our Safe RL training hypothesis.

However, changes in the MPC prediction model parametrization require a corresponding adjustment in the cost function parametrization. In contrast to manual tuning, our approach facilitates automated pre-optimization of the cost function weights coupled with RL training. In future work, we aim to investigate an automated cost function parametrization approach for various prediction model parameters.

References

[1] V. Ramasamy, R. K. Sidharthan, R. Kannan, and G. Muralidharan, "Optimal tuning of model predictive controller weights using genetic algorithm with interactive decision tree for industrial cement kiln process," *Processes*, vol. 7, no. 12, p. 938, 2019.

[2] L. L. Rodrigues, A. S. Potts, O. A. Vilcanqui, and A. J. Sguarezi Filho, "Tuning a model predictive controller for doubly fed induction generator employing a constrained genetic algorithm," *IET Electric Power Applications*, vol. 13, no. 6, pp. 812–819, 2019.

[3] A. Kapnopoulos and A. Alexandridis, "A cooperative particle swarm optimization approach for tuning an mpc-based quadrotor trajectory tracking scheme," *Aerospace Science and Technology*, vol. 127, p. 107725, 2022.

[4] D. Piga, M. Forgione, S. Formentin, and A. Bemporad, "Performance-oriented model learning for data-driven mpc design," *IEEE control systems letters*, vol. 3, no. 3, pp. 577–582, 2019.

[5] A. Gharib, D. Stenger, R. Ritschel, and R. Voßwinkel, "Multi-objective optimization of a path-following mpc for vehicle guidance: A bayesian optimization approach," in *2021 European Control Conference (ECC)*. IEEE, 2021, pp. 2197–2204.

[6] F. Sorourifar, G. Makrygirgos, A. Mesbah, and J. A. Paulson, "A data-driven automatic tuning method for mpc under uncertainty using constrained bayesian optimization," *IFAC-PapersOnLine*, vol. 54, no. 3, pp. 243–250, 2021.

[7] J. A. Paulson, K. Shao, and A. Mesbah, "Probabilistically robust bayesian optimization for data-driven design of arbitrary controllers with gaussian process emulators," in *2021 60th IEEE Conference on Decision and Control (CDC)*. IEEE, 2021, pp. 3633–3639.

[8] A. Rupenyan, M. Khosravi, and J. Lygeros, "Performance-based trajectory optimization for path following control using bayesian optimization," in *2021 60th IEEE Conference on Decision and Control (CDC)*. IEEE, 2021, pp. 2116–2121.

[9] G. Makrygiorgos, A. D. Bonzanini, V. Miller, and A. Mesbah, "Performance-oriented model learning for control via multi-objective bayesian optimization," *Computers & Chemical Engineering*, vol. 162, p. 107770, 2022.

[10] B. Zarrouki, C. Wang, and J. Betz, "Adaptive stochastic nonlinear model predictive control with look-ahead deep reinforcement learning for autonomous vehicle motion control," *arXiv preprint arXiv:2311.04303*, 2023.

[11] E. Bøhn, S. Moe, S. Gros, and T. A. Johansen, "Reinforcement learning of the prediction horizon in model predictive control," *IFAC-PapersOnLine*, vol. 54, no. 6, pp. 314–320, 2021.

[12] E. Bøhn, S. Gros, S. Moe, and T. A. Johansen, "Optimization of the model predictive control meta-parameters through reinforcement learning," *Engineering Applications of Artificial Intelligence*, vol. 123, p. 106211, 2023.

[13] M. Mehndiratta, E. Camci, and E. Kayacan, "Automated tuning of nonlinear model predictive controller by reinforcement learning," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2018, pp. 3016–3021.

[14] B. Zarrouki, V. Klös, N. Heppner, S. Schwan, R. Ritschel, and R. Voßwinkel, "Weights-varying mpc for autonomous vehicle guidance: a deep reinforcement learning approach," in *2021 European Control Conference (ECC)*. IEEE, 2021, pp. 119–125.

[15] B. Zarrouki, "Reinforcement learning of model predictive control parameters for autonomous vehicle guidance," *Master's thesis*, 2020.

[16] K. P. Wabersich and M. N. Zeilinger, "Performance and safety of bayesian model predictive control: Scalable model-based rl with guarantees," *arXiv preprint arXiv:2006.03483*, 2020.

[17] F. Berkenkamp, A. P. Schoellig, and A. Krause, "Safe controller optimization for quadrotors with gaussian processes," in *2016 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2016, pp. 491–496.

[18] F. Berkenkamp, M. Turchetta, A. Schoellig, and A. Krause, "Safe model-based reinforcement learning with stability guarantees," *Advances in neural information processing systems*, vol. 30, 2017.

[19] B. Zarrouki, C. Wang, and J. Betz, "A stochastic nonlinear model predictive control with an uncertainty propagation horizon for autonomous vehicle motion control," *arXiv preprint arXiv:2310.18753*, 2023.

[20] B. Zarrouki, J. Nunes, and J. Betz, "R²nmpc: A real-time reduced robustified nonlinear model predictive control with ellipsoidal uncertainty sets for autonomous vehicle motion control," *arXiv preprint arXiv:2311.06420*, 2023.

[21] D. Milios, R. Camoriano, P. Michiardi, L. Rosasco, and M. Filippone, "Dirichlet-based Gaussian Processes for Large-scale Calibrated Classification," 2018.

[22] M. Emmerich, K. C. Giannakoglou, and B. Naujoks, "Single- and multiobjective evolutionary optimization assisted by Gaussian random field metamodels," *IEEE Transactions on Evolutionary Computation*, vol. 10, no. 4, pp. 421–439, 2006.

[23] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017.

[24] P. Karle, T. Betz, M. Bosk, F. Fent, N. Gehrke, M. Geisslinger, L. Gressenbuch, P. Hafemann, S. Huber, M. Hübner, S. Huch, G. Kaljavesi, T. Kerbl, D. Kulmer, T. Mascetta, S. Maierhofer, F. Pfab, F. Rezabek, E. Rivera, S. Sagmeister, L. Seidlitz, F. Sauerbeck, I. Tahiraj, R. Trauth, N. Uhlemann, G. Würsching, B. Zarrouki, M. Althoff, J. Betz, K. Bengler, G. Carle, F. Diermeyer, J. Ott, and M. Lienkamp, "Edgar: An autonomous driving research platform – from feature development to real-world application," *arXiv preprint arXiv:2309.15492*, 2023.

[25] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021. [Online]. Available: http://jmlr.org/papers/v22/20-1364.html

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 01,2026 at 15:46:12 UTC from IEEE Xplore. Restrictions apply.